TAPAS: A Case Study on Topology Aware Multi-PIM in Model Parallelism

Yun Seok Kim
School of Electrical and Electronic Engineering
Yonsei University
Seoul, Korea
yunseok.kim@yonsei.ac.kr

Won Woo Ro
School of Electrical and Electronic Engineering
Yonsei University
Seoul, Korea
wro@yonsei.ac.kr

***Abstract*—The growing scale of large language models (LLMs) intensifies the need for efficient inference accelerators, particularly for attention mechanisms dominated by memory-bound matrix–vector multiplications (GEMV). Processing-in-Memory (PIM) offers a compelling solution by reducing off-chip data transfers and improving bandwidth utilization. Yet, as the key–value (KV) cache grows with longer sequences, a single PIM device becomes insufficient, necessitating Multi-PIM systems. This work investigates the architectural trade-offs of Multi-PIM environments, focusing on tensor and pipeline parallelism, topology-aware data mapping, and inter-PIM communication strategies. Through simulations using AiM Simulator and Booksim2, we demonstrate that while Multi-PIM significantly enhances GEMV utilization, its scalability critically depends on careful coordination of parallelism and interconnect topology.**

***Index Terms*—Large Language Model, Process-In-Memory, Multi-PIM, Topology, Model Parallelism**

I. Introduction

The inference performance of modern Large Language Model (LLM) is increasingly dictated by the efficiency of Attention mechanism [1]. Unlike training, which relies on compute-intensive matrix-matrix multiplications (GEMM), inference proceeds with matrix-vector multiplications (GEMV), where each new query must attend to all stored keys and values. This workload exhibits low arithmetic intensity, making memory access the critical bottleneck. Process-in-Memory (PIM) architectures directly address this challenge by embedding lightweight compute units within memory stacks, thereby reducing data movement and improving bandwidth efficiency. However, as LLMs expand in model size and context length, the key-value (KV) cache and query traffic surpass the storage and throughput limits of a single PIM device.

To accommodate these requirements, Multi-PIM systems distribute computation and storage across multiple devices connected through on-package or board-level interconnects. While this scaling increases aggregate capacity, it also introduces new system-level challenges such as workload imbalance, inter-PIM communication overheads, and topology-induced latency. Arbitrary data placement can exacerbate these inefficiencies, limiting the benefits of Multi-PIM. In this study, we explore how Tensor Parallelism (TP) and Pipeline Parallelism (PP) can be applied within Multi-PIM systems, and how topology-aware data mapping can mitigate imbalance and improve utilization.

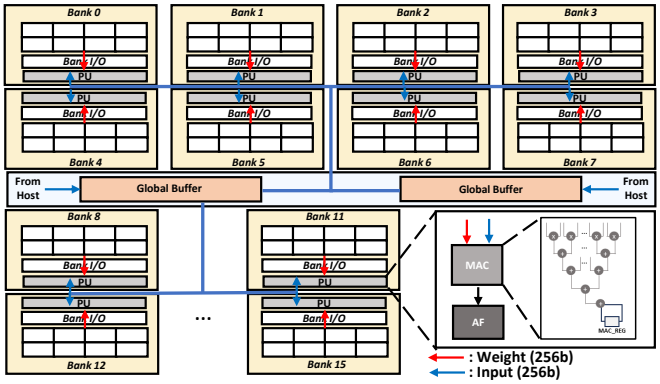


Fig. 1: Baseline PIM Architecture. Host reads or writes through the Global Buffer to perform Multiplication and Accumulation in memory. Commands support both all-bank and single-bank operations. Bit-width of 256b enables 16 elements of BF16 per Processing Unit. After MAC operation, the results are passed on to the activation function. MAC/AF registers were omitted for simplicity.

II. Background

Attention. Given query, key, and value matrices $Q, K, V \in \mathbb{R}^{n \times d}$, the standard scaled dot-product Attention computes $O = \mathrm{Softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$ where the Attention score matrix $S = \frac{1}{\sqrt{d}} QK^\top \in \mathbb{R}^{n \times n}$ is explicitly formed. During training, this formulation is implemented as large GEMM, which can be efficiently handled by GPUs and other accelerators. However, inference proceeds differently: only a single new query vector arrives at each decoding step, and this vector must be multiplied with all previously stored keys. As a result, the core operation reduces from a GEMM to a series of GEMV.

This shift places the computational burden not on arithmetic throughput but on memory access. Specifically, KV cache, which stores all past tokens' representations, grows linearly with sequence length. Each new query requires fetching an increasingly large number of key and value vectors from this cache, turning it into the dominant bottleneck in Attention

inference. The compute units themselves remain underutilized, waiting for memory fetches to complete, making memory bandwidth—not FLOPs—the critical limiting factor.

DRAM-based PIM. Fig. 1 describes the baseline PIM performing GEMV at all-bank level and a detailed stages in Multiplication and Accumulation (MAC) operation. Arrows in red are the weights while those in blue are the inputs. DRAM-based PIM alleviates memory bottlenecks by moving computation directly into the memory stack. In fixed-function designs optimized for LLM inference, where GEMV dominates the workload, each DRAM bank integrates a lightweight MAC unit to perform parallel dot-product operations. After MAC results are stored in MAC registers, the processing unit performs Activation Function (AF) computation and stores the results in activation function registers. Control overhead is minimized through pre-generated PIM commands and limited conditional logic, keeping area costs low. For example, AiMX [2] incorporates a 16-element vector MAC per bank, with inputs supplied from a shared 2 KB Global Buffer (GB). Higher-level operations such as Softmax are executed externally.

Model Parallelism. Decoding in transformer inference is GEMV-dominant, making memory access the key bottleneck. Each query must fetch the entire KV cache, and as models and sequence lengths grow, the cache scales proportionally, quickly exceeding the capacity of a single PIM device. This necessitates distributing storage and computation across multiple PIMs. Grouping Multi-PIMs in a system can be an option, but new bottlenecks(e.g., topology and data mapping) arise as we will see in III. To mitigate this, we apply the concept of Model Parallelism.

Model parallelism is a strategy where a single large neural network model is split across multiple devices so that each device is responsible for a portion of the model. Model Parallelism has two common forms which are TP and Pipeline Parallelism. TPpartitions the parameters of a single workload into smaller pieces that are distributed across multiple processors and executed simultaneously. In contrast, PP partitions the model into sequential stages, each assigned to a different processor, so that different stages can be executed in parallel on different inputs. In our case, the splitting of a decoder block and allocating them to each PIM device is TP and placing multiple decoder blocks within a single PIM device is Pipeline Parallelism. Detailed depiction of how TP and PP is applied to Multi-PIM was measured with MultiPIM [3] and can be found in Fig. 2 and will be further discussed in IV.

III. Motivation

Workload Imbalance upon PIM Topology. From the previous section, we discussed why Multi-PIM system is required for recent LLM workloads and what methods can be applied to such environment. However, naively placing Multi-PIM with the absence of careful consideration about topology can lead to workload imbalance. Fig. 3 describes how mesh and dragonfly topology without appropriate data mapping can cause unevenly distributed active cycles among processing units in a PIM device. The dotted line marks the boundary between two PIM devices and a total of four PIM was tested. The graph represents 4 PIM devices' core activation in the decode stage with mesh and dragonfly topology, while small and large represents the number of parameters. It shows that not only average cycles vary among PIMs but very few cores were active during the workload. It is clear that PIM cores are unevenly utilized in such workload and topology. This shows that topology must be taken into account depending on the workload.

Inter-PIM Data Mapping Another challenge arises in data placement, especially when utilizing Model Parallelism. As mentioned in II, TP was applied to head-wise partitioning and PP was applied to batching sequential decoder blocks in a PIM device. However, applying arbitrarily chosen topology can cause inefficient data transfer between PIMs. For example, in PP environment from Fig. 2, if PIM1 had connection to PIM3 or PIM4, intermediate results would need to traverse multiple hops, incurring additional latency and bandwidth pressure. This highlights that in PP environments, aligning the inter-PIM topology with logical execution order is critical. This work explores these architectural trade-offs in Multi-PIM environments. Using AiM Simulator [3] and Booksim2 [4], we analyze how topology and inter-PIM transfer strategies affect utilization and performance. Our study highlights that although Multi-PIM architectures inherently provide higher GEMV throughput, achieving scalable acceleration requires careful co-design of topology and data placement.

IV. Design Space Exploration

In order to observe the impact of intra-PIM data mapping, we utilize AiM Simulator, a trace-driven simulator for AiM architecture, to simulate the inter-bank data transfer and channel utilization. The configurations are set to the GDDR6 configuration. Also, we utilize Booksim2, a cycle-accurate interconnection network simulator, to simulate various topologies and measure their latencies. The LLM model which we used throughout the simulation was Llama-2 7B with single batch, 128 hidden dimension, 32 decode layers, 32 heads and sequence length of 1024. Meanwhile, latencies were calculated according to the cycles retrieved from AiM Simulator and Booksim2.

Two scenarios of TP and PP were simulated each with different topologies. In TP environment, each decoding block was processed by four PIM devices and each PIM device computed 8 heads, since there were 32 heads. After each PIM device had output its result, it was sent to the host for concatenation and the concatenated result was sent back to the next decoding block. The generated trace includes the following stages: First, each PIM loads Query vector to GB and KV cache to banks via write all bank command using 8 channels(1 channel per head). After loading Q, K and V, logit calculation is done with MAC operation on all bank and sent to host by reading MAC registers. Finally, the result of softmax is sent to GB and MAC operations on all bank with V for the final output. We presumed that the concatenation of heads

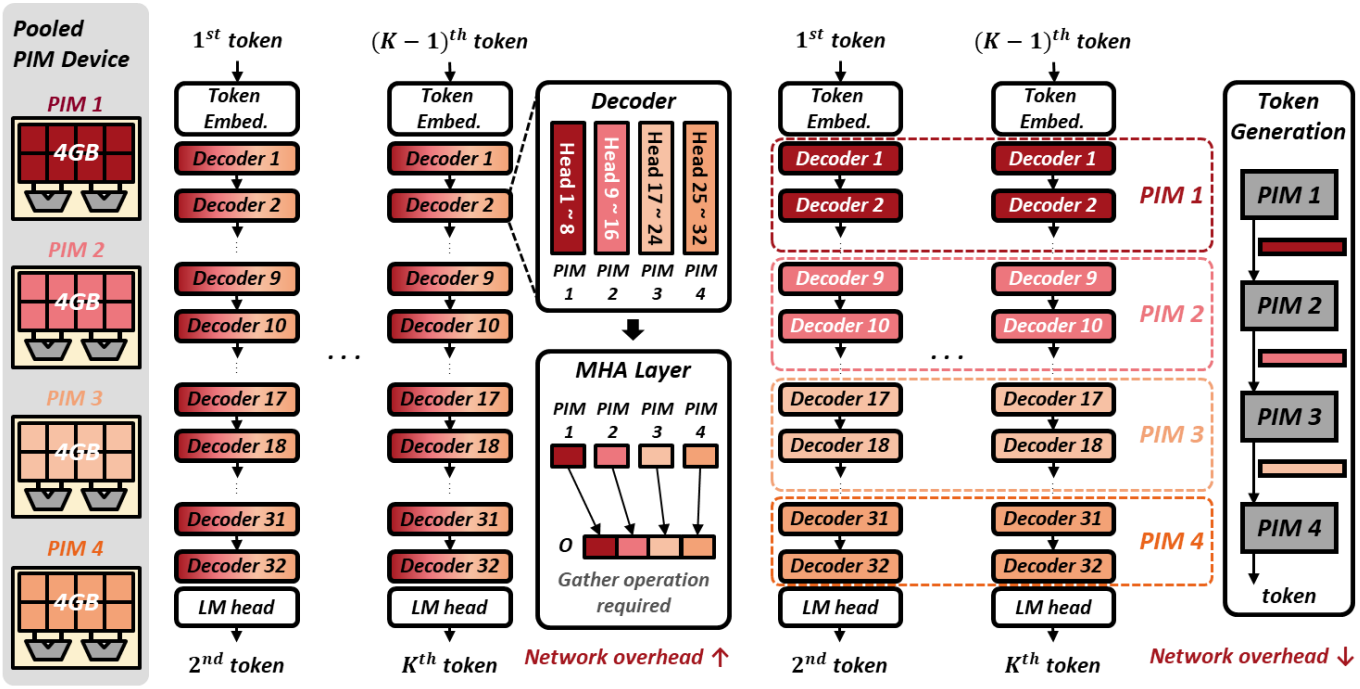


Fig. 2: Data mapping of LLM execution utilizing Model Parallelism across multi-PIM. The assumption is that the model is composed of 32 decoding layers, 32 heads and each PIM has the memory capacity of 4GB. Each token goes through 32 decoding blocks to create the next token.

occurred at one of the PIMs. Since TP requires inter-PIM data transfer for concatenation, it was simulated on all dragonfly, mesh and torus. But the fact that frequent concatenation cause additional latency was considered in evaluation.

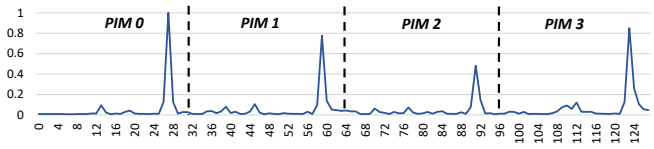
In PP environment, each PIM device processed 8 decoding blocks and on the final decoding block of PIM, the result was sent to the next PIM device. Since each PIM device had to process all 32 heads, this resulted in higher memory consumption due to increase of KV cache but less inter-PIM transfer was expected. The generated trace was the same, except this time it utilized all 32 channels. However, torus was the only valid option because decoder blocks needed to be processed sequentially.

Fig. 4 presents the Booksim2 simulation results for three interconnect topologies: Mesh, Dragonfly, and Torus. Each configuration consisted of four routers with one node per router. Although Mesh and Torus appear structurally similar, their distinction lies in link directionality—Mesh employs bidirectional links, whereas Torus adopts unidirectional connections. We sweep the injection rate from 0.1 to 0.2 in increments of 0.05; this range is calibrated to remain below the congestion knee while still reflecting the burstiness of LLM decoding traffic. Within this regime, Dragonfly consistently yielded the lowest latency, confirming its superior scalability. Torus showed slightly higher latency than Mesh, but the gap remained modest (8–10%) within the practical injection range of 0.1–0.15.

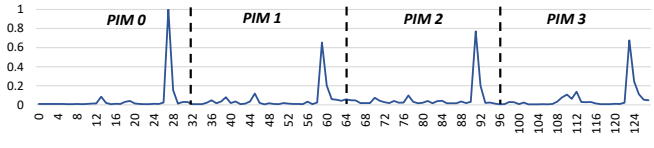
Fig. 5 reports channel-level activation cycles under TP and PP in all 32 decoding blocks. Compared with the baseline imbalance in Fig. 3, both TP and PP achieved more uniform utilization across PIM devices. Notably, TP delivered a 3.36× average per-channel speedup over PP, attributed to the reduced number of heads assigned per device. In PP, each PIM processed all heads, causing proportional growth in KV cache size, MAC operations, and host transfers for softmax computation. These results highlight the trade-off: TP offers finer-grained workload partitioning but introduces higher inter-PIM communication overhead, whereas PP lowers communication demands at the cost of larger per-device memory pressure.

V. Conclusion

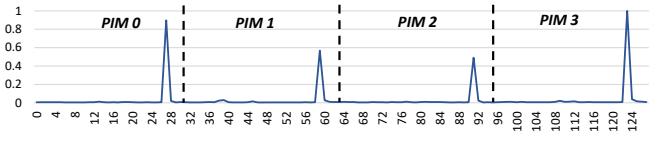
In this paper, we explored the appropriate Multi-PIM architectures for LLM inference and found that it's a promising path toward scalable acceleration of attention in LLM inference, but their success hinges on joint optimization of parallelism strategies and interconnect topology. By integrating Tensor



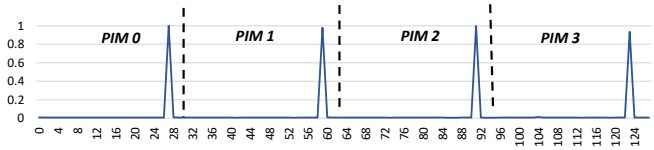
(a) Small decode mesh



(b) Large decode mesh



(c) Small decode dragonfly



(d) Large decode dragonfly

Fig. 3: Workload imbalance in decoding stage. Given that matrix parameters are composed as [Number of heads, Sequence Length, Hidden Dimension], small decode means [32,1,128]x[32,128,128] and large decode means [32,1,128]x[32,1024,128].

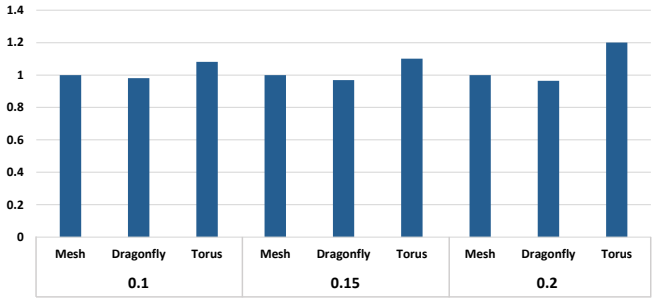


Fig. 4: Average Packet Latency on Various Topologies and Injection Rate. Injection rate was tested on 0.1, 0.15 and 0.2 on three topologies(mesh, dragonfly and torus). Results were normalized to the results of mesh.

Parallelism and Pipeline Parallelism with topology-aware mapping, we demonstrate improved balance across devices and reduced communication latency, discovering the possibilities of substantial performance gains compared to naïve deployments. These results underscore the importance of holistic co-design in Multi-PIM systems, where architecture, data placement, and parallel execution must be orchestrated together to fully realize the potential of memory-centric acceleration.

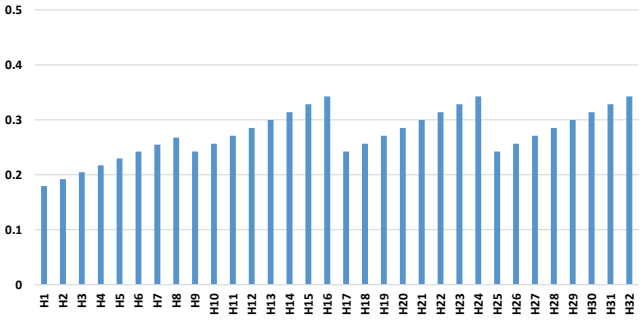


Fig. 5: Normalized TP latency per head. PP latency is not shown, as TP latency was normalized with respect to PP. Each channel was exclusively assigned to a single head only. The latency of TP corresponds to the average over mesh, dragonfly and torus, while PP latency is the result of torus only.

VI. Acknowledgment

This work was supported by Institute of Information & communication Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2025-0-00441, Memory-Centric Architecture Using the Reconfigurable PIM Devices). Won Woo Ro is the corresponding author.

References

[1] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez A N, Kaiser, and Polosukhin I Attention is all you need Proceedings of the 31st International Conference on Neural Information Processing Systems 2017 6000-6010

[2] S. Lee et al., "A 1ynm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications," 2022 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2022, pp. 1-3

[3] C. Yu, S. Liu and S. Khan, "MultiPIM: A Detailed and Configurable Multi-Stack Processing-In-Memory Simulator," in IEEE Computer Architecture Letters, vol. 20, no. 1, pp. 54-57, 1 Jan.-June 2021, doi: 10.1109/LCA.2021.3061905.

[4] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi Iyer, and Reetuparna Das. PIM is All You Need: A CXL-Enabled GPU-Free System for LLM Inference, In 2025 International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)

[5] Nan Jiang, Daniel U. Becker, George Michelogiannakis, James Balfour, Brian Towles, John Kim and William J. Dally. A Detailed and Flexible Cycle-Accurate Network-on-Chip Simulator. In Proceedings of the 2013 IEEE International Symposium on Performance Analysis of Systems and Software, 2013.